

POLITECNICO DI MILANO

Compressed Sensing using Generative Models

Matteo Forlivesi

Renato Gallicola

1 SUMMARY

1.1 INTRODUCTION

This experiment aims to evaluate the performance of compressed sensing using two models on the MNIST dataset: a Variational Auto-encoder (VAE) and a Generative Adversarial Network (GAN). The focus is on image reconstruction from compressed measurements. The MNIST dataset consists of 60,000 training images and 10,000 test images of handwritten digits, each of size 28x28 pixels.

1.2 MODELS AND METHODOLOGY

We employed two approaches to reconstruct the images from compressed measurements:

1. **Variational Auto-encoder (VAE):** This model is designed to generate images similar to those in the training set. The VAE compresses the input image into a lower-dimensional latent space and then reconstructs the image from this representation.
2. **Generative Adversarial Network (GAN):** This model is composed of two competing networks, a generative one and a discriminating one. They are trained to minimize their losses:
 - The generator tries to generate images from a random latent vector.
 - The discriminator tries to discern between real images and generated ones.

For both models, we used a random Gaussian matrix for initial measurements. The compression levels varied from 20 to 770 measurements per image, allowing us to observe the reconstruction quality across different levels of compression.

1.3 ALGORITHM EXPLANATION

This section will explain the algorithm utilizing a generative model for compressed sensing.

Enhancements include training for 1000 epochs and 10 random restarts to improve convergence and error minimization.

1.4 THE MNIST DATASET

The MNIST (Modified National Institute of Standards and Technology) dataset is a large collection of handwritten digits commonly used for training and evaluating image processing systems. It contains 60,000 training images and 10,000 test images, each being a gray scale image of size 28×28 pixels. Each image is labeled with a digit from 0 to 9, providing a diverse set of handwriting styles for developing robust digit recognition systems.

- **Size:** 60,000 training images and 10,000 test images.
- **Resolution:** Each image is 28×28 pixels, resulting in 784 features.
- **Classes:** 10 classes, representing digits 0 through 9.
- **Grayscale:** Pixel values range from 0 (black) to 255 (white).

The MNIST images were vectorized, resulting in 784-dimensional input vectors. Each pixel value was normalized to the range $[0, 1]$.

1.5 RESULTS

The reconstruction performance was evaluated using the mean squared error (MSE) between the original and reconstructed images. The following observations were made:

1. **VAE Performance:** The VAE achieved lower reconstruction errors with fewer measurements compared to traditional sparse recovery methods such as Lasso. For example, with 50 measurements, the VAE's performance was comparable to Lasso's performance with 400 measurements.
2. **GAN Performance:** The performance was slightly better than the VAE architecture. Reconstruction errors decreased as the number of measurements increased, demonstrating the model's capacity to capture more image details with more data. Exactly like VAE, the performance capped at around 400 measurements, gaining almost no benefit from sampling more.

1.6 CONCLUSION

The experiment demonstrated the efficacy of using deep learning models, particularly VAEs and GANs, for image reconstruction from compressed measurements. These models significantly reduce reconstruction error compared to traditional sparse recovery methods, especially at lower measurement levels. The findings highlight the potential of integrating compressed sensing techniques with deep learning to enhance image reconstruction quality in resource-constrained environments.

These results provide a promising direction for further research, particularly in optimizing measurement matrices and exploring other deep learning architectures for compressed sensing applications.

2 INTRODUCTION

Compressed sensing is a signal processing technique that enables the reconstruction of a signal from a small number of measurements. Given the measurement y , the measurements matrix $A \in \mathbb{R}^{m \times n}$ and the noise $\eta \in \mathbb{R}^m$, the goal is to estimate the vector $x^* \in \mathbb{R}^n$ such that:

$$y = Ax^* + \eta \tag{2.1}$$

Since $m \ll n$, this system is underdetermined, then it is not possible to find a unique solution. The Nyquist–Shannon sampling theorem requires a certain amount of samples to perfectly reconstruct a signal, so compressed sensing exploits some features of the signal to address this problem in a context where the conditions of the aforementioned theorem are not met.

The standard version of the algorithm assumes the signal to be k -sparse, meaning that it has only k coefficients different from zero when represented in some domain. This assumption allows the signal retrieval through the use of algorithms for solving minimization problems, such as LASSO for the L^1 norm minimization.

This experiment shifts from sparsity to structures derived from generative models, with the aim of assessing the goodness of these models when employed in compressed sensing. They feature a generator that maps vectors from a low dimensional to a high dimensional space, so from $z \in \mathbb{R}^k$ to $G(z) \in \mathbb{R}^n$. By training the generator, with high probability it will output vectors similar to the ones belonging to the dataset it has been trained on.

The idea behind the use of generative models in the context of compressed sensing is that we expect the vector x^* to be close to some $G(z)$, which can be found with an optimization algorithm based on gradient descent, that will be later discussed in detail.

3 MODELS

3.1 DCGAN

Generative Adversarial Networks (GANs) are a class of machine learning frameworks designed by Ian Goodfellow and his colleagues in 2014. GANs have revolutionized the field of generative modeling, allowing for the creation of highly realistic synthetic data. This section will explore the fundamental concepts, structure, and applications of Deep Convolutional Generative Adversarial Networks (DCGANs).

3.1.1 INTRODUCTION TO DCGANS

DCGANs consist of two neural networks: the generator and the discriminator, which compete to minimize their respective loss functions. The generator aims to create data that is indistinguishable from real data, while the discriminator attempts to differentiate between real and generated data. This adversarial process continues until the generator produces data that the discriminator cannot reliably distinguish from real data. DCGANs leverage convolutional neural networks (CNNs) to improve the quality of generated images.

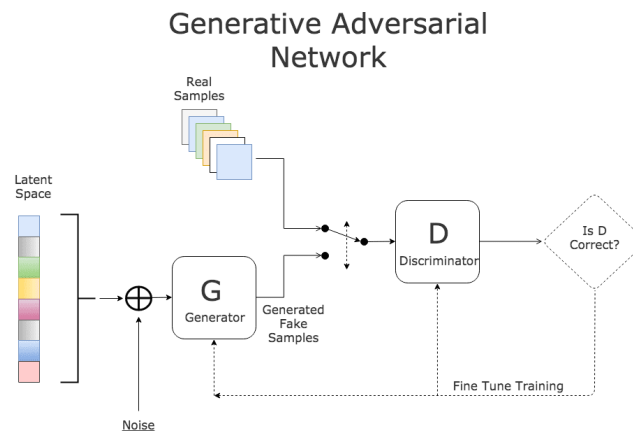


Figure 3.1: Structure of a Generative Adversarial Network, picture from: <https://www.kdnuggets.com/2017/01/generative-adversarial-networks-hot-topic-machine-learning.html>

3.1.2 COMPONENTS OF DCGANS

The two primary components of DCGANs are:

GENERATOR

The generator, G , is a convolutional neural network that takes random noise, z , from a latent space and transforms it into a synthetic data sample, $G(z)$. The objective of the generator is to produce data that is as close as possible to the real data distribution. The generator's loss function is designed to minimize the likelihood of the discriminator correctly identifying the generated data as fake.

The actual structure of the generator (inspired by the standard DCGAN presented here: <https://arxiv.org/pdf/1511.06434>):

Model: "generator"		
Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 1152)	24192
reshape (Reshape)	(None, 3, 3, 128)	0
conv2d_transpose (Conv2DTranspose)	(None, 6, 6, 128)	262272
leaky_re_lu_3 (LeakyReLU)	(None, 6, 6, 128)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 14, 14, 256)	524544
leaky_re_lu_4 (LeakyReLU)	(None, 14, 14, 256)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 28, 28, 512)	2097664
leaky_re_lu_5 (LeakyReLU)	(None, 28, 28, 512)	0
conv2d_3 (Conv2D)	(None, 28, 28, 1)	12801
...		
Total params: 2,921,473		
Trainable params: 2,921,473		
Non-trainable params: 0		

Figure 3.2: Structure of a Deep Convolutional Generative Adversarial Network Generator

DISCRIMINATOR

The discriminator, D , is another convolutional neural network that evaluates data and assigns a probability that a given sample comes from the real data distribution rather than the generator. The discriminator's loss function aims to maximize the likelihood of correctly classifying real and fake data.

The actual structure of the discriminator (inspired by the standard DCGAN presented here: <https://arxiv.org/pdf/1511.06434>):

Model: "Discriminator"		
Layer (type)	Output Shape	Param #
=====		
conv2d (Conv2D)	(None, 14, 14, 64)	1088
leaky_re_lu (LeakyReLU)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 7, 7, 128)	131200
leaky_re_lu_1 (LeakyReLU)	(None, 7, 7, 128)	0
conv2d_2 (Conv2D)	(None, 4, 4, 128)	262272
leaky_re_lu_2 (LeakyReLU)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dropout (Dropout)	(None, 2048)	0
dense (Dense)	(None, 1)	2049
=====		
Total params: 396,609		
Trainable params: 396,609		
Non-trainable params: 0		

Figure 3.3: Structure of a Deep Convolutional Generative Adversarial Network Discriminator

3.1.3 TRAINING PROCESS

The training process for DCGANs involves optimizing the following value function, $V(G, D)$, which is a minimax game between G and D :

$$\min_G \max_D V(G, D) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))] \quad (3.1)$$

The training of DCGANs involves alternating between updating the discriminator and the generator. The discriminator is trained to maximize its classification

accuracy, while the generator is trained to deceive the discriminator. This iterative process can be summarized in the following steps:

1. Sample a batch of real data and a batch of random noise.
2. Update the discriminator by maximizing its loss function with both real and generated data.
3. Sample a new batch of random noise.
4. Update the generator by minimizing its loss function, which depends on the discriminator's performance.

3.2 VAE

3.2.1 INTRODUCTION TO VAE

The **Variational Auto-encoder** (VAE) is an artificial neural network whose encoder and decoder are connected through a probabilistic latent space, corresponding to the parameters of a variational distribution.

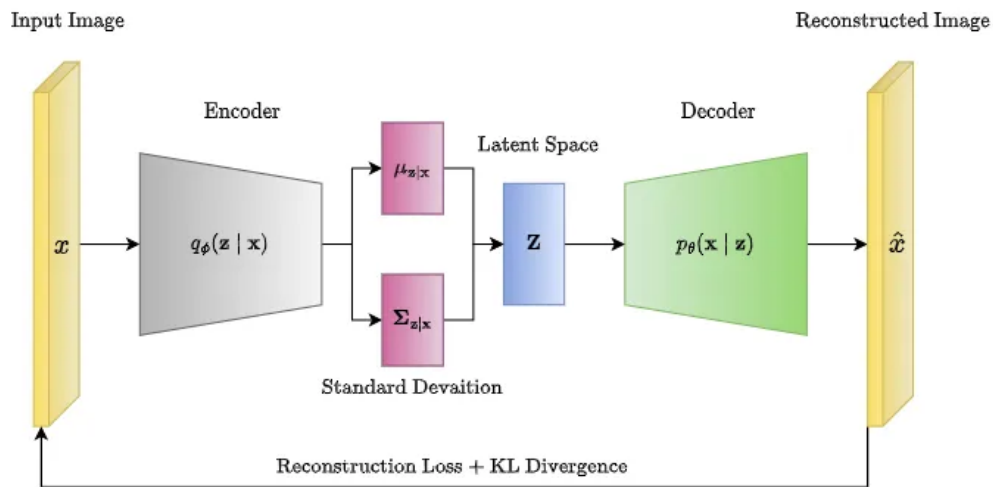


Figure 3.4: Structure of a Variational Auto-encoder, picture from <https://medium.com/@rushikesh.shende/autoencoders-variational-autoencoders-vae-and-beta-vae-ceba9998773d>

The encoder receives as input a tensor representing an image extracted from a dataset, and it maps it into a distribution within the latent space, parametrized by a mean vector μ and a standard deviation vector σ .

The decoder, instead, maps from the latent space back to the input space.

The VAE's parameters are optimized via gradient descent, however the sampling of the latent variable z introduces stochasticity, making the gradient of the loss with respect to the model parameters non-differentiable. A technique known as **Reparameterization Trick** is employed to address this problem, which expresses the sampling operation in a way that separates the deterministic and stochastic parts, enabling gradient flow through the deterministic part. So instead of directly sampling z as $z \sim N(\mu, \sigma^2)$, which is non-differentiable with respect to μ and σ , it sample an auxiliary variable ϵ from a standard normal distribution $N(0, I)$, and then it expresses z as $z = \mu + \sigma \cdot \epsilon$, which is now a differentiable function of μ and σ .

3.2.2 COMPONENTS OF VAE

The two primary components of DCGANs are:

ENCODER

The encoder's architecture consists of four 2D convolution layers, one flatten layer and one dense layer. The input received by this module, which is a 1-channel 28×28 image, passes through the convolutional layers – using 32, 64, 64 and 64 filters respectively – to extract features, then the feature maps is flattened and its dimensionality is reduced using a dense layer; finally the encoder outputs the mean and standard deviation for the latent space distribution and samples the latent vector z .

Model: "encoder"			
Layer (type)	Output Shape	Param #	Connected to
encoder_input (InputLayer)	[(None, 28, 28, 1)]	0	[]
conv2d (Conv2D)	(None, 28, 28, 32)	320	['encoder_input[0][0]']
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18496	['conv2d[0][0]']
conv2d_2 (Conv2D)	(None, 14, 14, 64)	36928	['conv2d_1[0][0]']
conv2d_3 (Conv2D)	(None, 14, 14, 64)	36928	['conv2d_2[0][0]']
flatten (Flatten)	(None, 12544)	0	['conv2d_3[0][0]']
dense (Dense)	(None, 32)	401440	['flatten[0][0]']
z_mu (Dense)	(None, 2)	66	['dense[0][0]']
z_sigma (Dense)	(None, 2)	66	['dense[0][0]']
z (Lambda)	(None, 2)	0	['z_mu[0][0]', 'z_sigma[0][0]']
=====			
Total params: 494,244			
Trainable params: 494,244			
Non-trainable params: 0			

Figure 3.5: Structure of a VAE Encoder

DECODER

The decoder's architecture consists of a dense layer, a reshape layer and two 2D convolution layers. The input of the module is the latent vector that, through the dense and reshape layers, it is transformed back into a tensor, matching the shape of the last convolution layer of the encoder; finally the convolution layers generate the reconstructed image with the same number of channels as the original input image.

Model: "decoder"		
Layer (type)	Output Shape	Param #
decoder_input (InputLayer)	[(None, 2)]	0
dense_1 (Dense)	(None, 12544)	37632
reshape (Reshape)	(None, 14, 14, 64)	0
conv2d_transpose (Conv2DTranspose)	(None, 28, 28, 32)	18464
decoder_output (Conv2DTranspose)	(None, 28, 28, 1)	289
=====		
Total params: 56,385		
Trainable params: 56,385		
Non-trainable params: 0		

Figure 3.6: Structure of VAE Decoder

3.2.3 TRAINING PROCESS

The optimization of VAE is achieved through the maximization of the **Evidence Lower Bound** (ELBO), consisting of two terms: the **Reconstruction Loss** and the **Regularization Term** (Kullback–Leibler Divergence).

Given the input data x , the latent variable z , the encoder network $q_\phi(z|x)$, the decoder network $p_\theta(x|z)$, the prior distribution over the latent variables $p(z)$, the reconstruction loss $\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]$, and the KL divergence $KL(q_\phi(z|x) || p(z))$, the loss is computed as:

$$L(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - KL(q_\phi(z|x) || p(z)) \quad (3.2)$$

The VAE is trained with a custom loop, consisting of three main steps: **forward pass**, **loss calculation** and **backward pass**. During the forward pass, the input data is passed through the encoder to get the mean and log vari-

ance of the latent distribution, then the latent vector z is sampled using the reparametrization trick, and finally z is passed through the decoder to get the reconstructed data. The loss calculation step computes the reconstruction loss between the input data and the reconstructed data, the KL divergence, and then sums them up to compute the total loss. Finally, in the backward pass the total loss is backpropagated to update the parameters of the encoder and decoder.

The model is trained for 100 epochs using a batch size of 100 and an Adam optimizer with learning rate of 0.001.

To test the goodness of the model in the context of compress sensing, two versions of the VAE have been generated: one with the dimension of the latent vector equal to 20, and the other equal to 30. After using both of them, it's possible to conclude that they lead to good results, as shown in the Results section.

3.3 MODEL RESULTS

The following graph shows the output mean vector produced by the encoder with the test set provided by the MNIST dataset. For the sake of simplicity, this result is obtained with a latent dimension equal to 2.

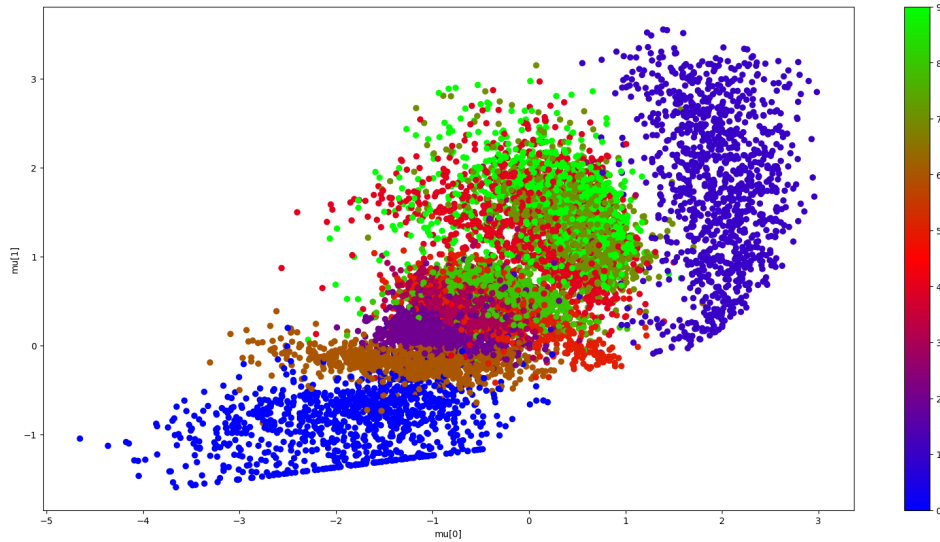


Figure 3.7: Plotted output of the encoder, each color represents a specific digit

By observing the data in the graph, we can predict the behaviour of the decoder. Infact, if we assume it receives as input the vector $[-2, -1]$, we expect the output to depict with good precision the number 0, as represented by the blue color in the lateral colorbar. The image reconstruction is shown in the image below.

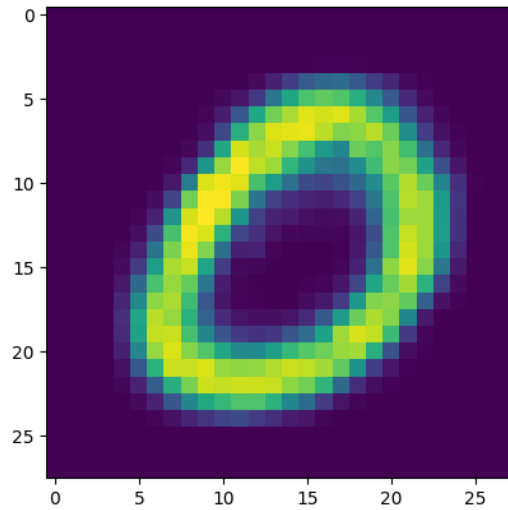


Figure 3.8: Output of the decoder for the input vector $[-2, -1]$

4 ALGORITHM EXPLANATION

Generative models offer a powerful approach for compressed sensing by leveraging the structure of the data distribution.

Independently from the above mentioned models, a generative model is defined by a deterministic function $G : \mathbb{R}^k \rightarrow \mathbb{R}^n$ and a distribution P_Z over $z \in \mathbb{R}^k$. The model maps a low-dimensional latent vector z (decided by us as the latent dimension) to the high-dimensional sample space (the final generated image). The compressed sensing task involves finding a vector z in the latent space such that the corresponding generated sample $G(z)$ matches the observed measurements y .

The objective function to be minimized is:

$$\text{loss}(z) = \|AG(z) - y\|^2. \quad (4.1)$$

Optimization is performed using gradient descent on z with the Adam algorithm for optimizing descent. If G is differentiable, gradients of the loss with respect to z can be computed using back-propagation. The process involves iteratively updating z to minimize the measurement error. Upon convergence to $\hat{z}$, the reconstruction of x^* is given by $G(\hat{z})$.

The algorithm can be summarized as follows:

1. Initialize z from the distribution P_Z .
2. Compute the loss: $\text{loss}(z) = \|AG(z) - y\|^2$.
3. Use gradient descent to update z to minimize the loss.
4. Repeat until convergence.
5. The reconstructed signal is $\hat{x} = G(\hat{z})$.

This method leverages the data prior encoded in the generative model to achieve high-quality reconstructions with fewer measurements compared to traditional compressed sensing methods.

In order to improve convergence we employed training for a thousand epochs and ten random restarts. This way we could choose the attempt that presented the smallest error at the end of the approximation.

5 THE MNIST DATASET

The MNIST (Modified National Institute of Standards and Technology) dataset is a large collection of handwritten digits commonly used for training various image processing systems. It serves as a benchmark for evaluating the performance of machine learning algorithms. The dataset contains 60,000 training images and 10,000 test images, each of which is a gray scale image of size 28×28 pixels.

Each image in the MNIST dataset is labeled with the digit it represents, ranging from 0 to 9. The dataset is particularly valuable because it provides a diverse set of handwriting styles, allowing for the development of robust digit recognition systems.

- **Size:** The training set consists of 60,000 images, and the test set consists of 10,000 images.
- **Resolution:** Each image is 28×28 pixels, making a total of 784 features.
- **Classes:** There are 10 classes, corresponding to the digits 0 through 9.
- **Grayscale:** Images are in grayscale, with pixel values ranging from 0 (black) to 255 (white).

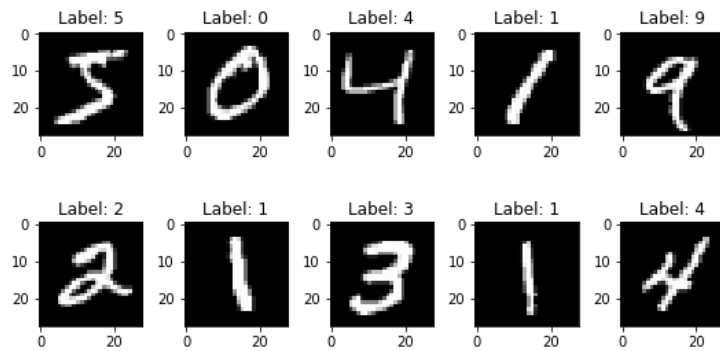


Figure 5.1: Sample of images from the MNIST dataset

6 RESULTS

We created two versions of both VAE and DCGAN using different lengths for the latent representation vector z . In one case, we used a latent vector of dimension 20, and in the other, a dimension of 30.

Given the trained models, we first visually examined the results of standard sensing at a compressed dimension of 300 ($M = 300$). All the reconstructions were obtained with 1000 epochs of training and 10 random restarts, selecting the best result.

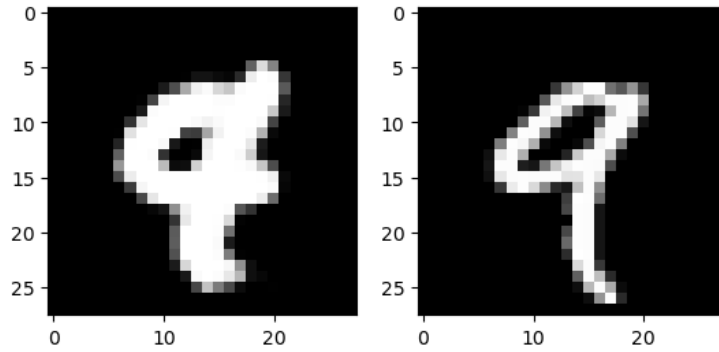


Figure 6.1: Sample image retrieval from 300 measurements using a latent dimension of 20 with a DCGAN model

6.1 BASELINE PERFORMANCE USING LASSO

We started by evaluating the baseline performance using LASSO for reconstruction. LASSO (Least Absolute Shrinkage and Selection Operator) assumes sparsity in the signal and solves the following optimization problem:

$$\hat{x} = \arg \min_x \|y - Ax\|_2^2 + \lambda \|x\|_1, \quad (6.1)$$

where y is the measurement vector, A is the measurement matrix, and λ is a regularization parameter. This method aims to find a sparse solution x that approximates the original signal.

From our experiments, we observed that the performance of LASSO is inadequate for this task until a substantial number of measurements (high M) is available. Even then, the reconstructions are often noisy.

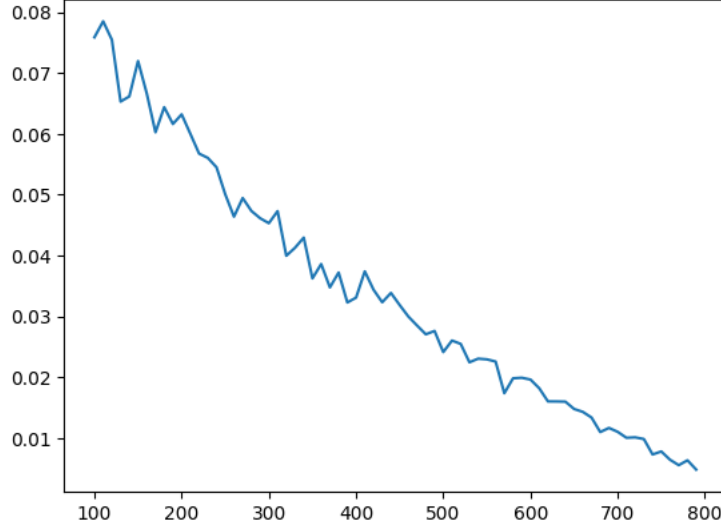


Figure 6.2: Error per pixel using LASSO for compressed sensing as a function of the compressed dimension M

The figure (6.3) illustrates the reconstructed images as the number of measurements M increases.

We observe that more than 400 measurements are needed for coherent reconstruction that effectively suppresses background noise, although 100 measurements are sufficient to start discerning the digit under the noise. This is achievable because the underlying shape is very simple. For more sophisticated datasets, achieving similar results would be more challenging.

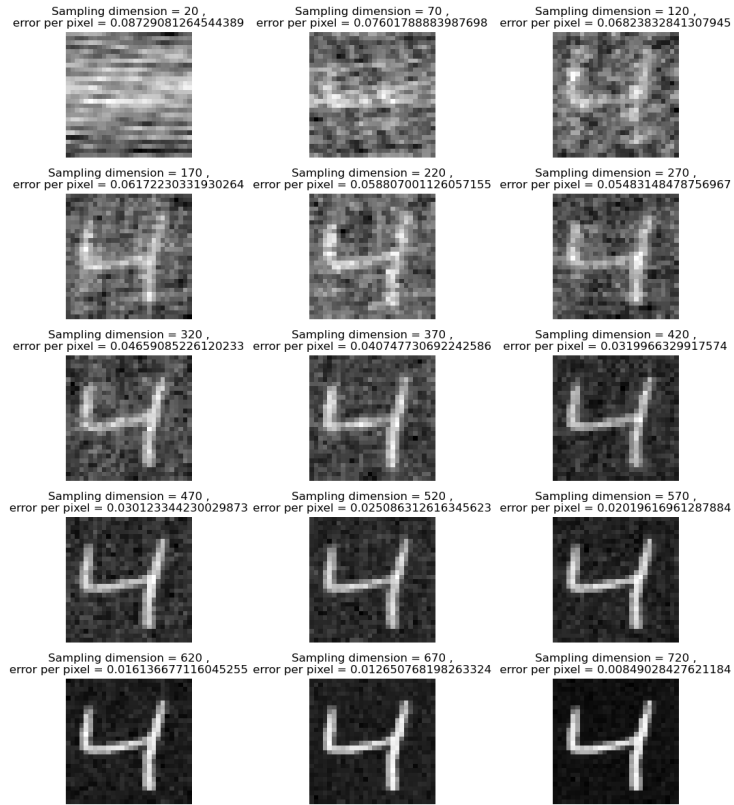


Figure 6.3: Reconstructions as a function of the increasing number of measurements M

6.2 IMPROVED PERFORMANCE USING GENERATIVE NETWORKS

Generative networks, in both VAE and DCGAN forms, significantly enhance performance, requiring far fewer measurements to achieve the same quality of reconstruction as LASSO. The following example (fig. 6.4) demonstrates the generation of images with a given number of measurements M .

We note that the generated number matches the original with as few as 20 measurements.

This significant improvement showcases the effectiveness of generative networks in compressed sensing, highlighting their ability to reconstruct high-quality images from minimal data.

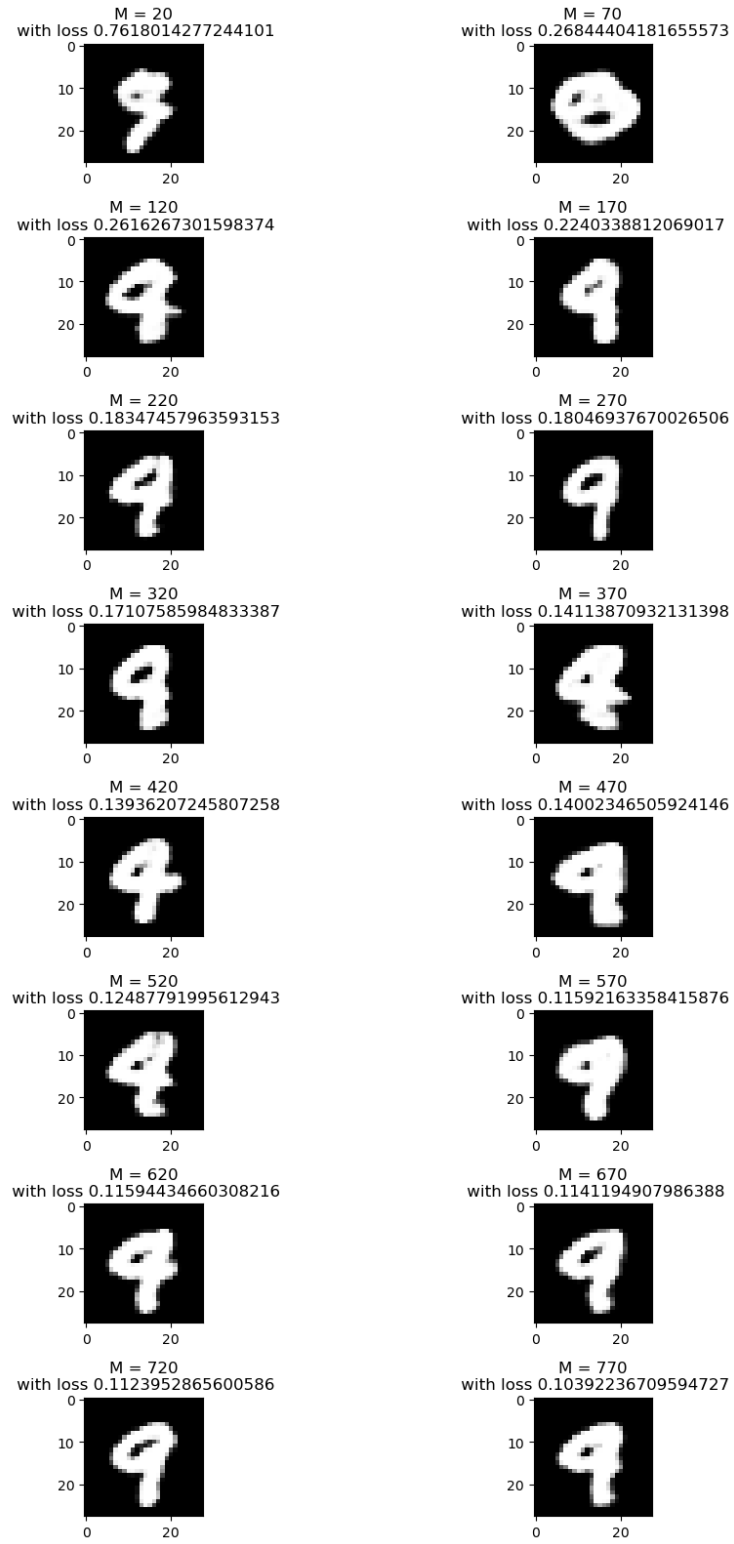


Figure 6.4: Sample images generated by a DCGAN with a latent vector of dimension 20

Furthermore, we can see how the four models (VAEs with 20 and 30 latent vector dimension and DCGANs with 20 and 30 latent dimension) stack against each other in the error per pixel for every increasing value of M .

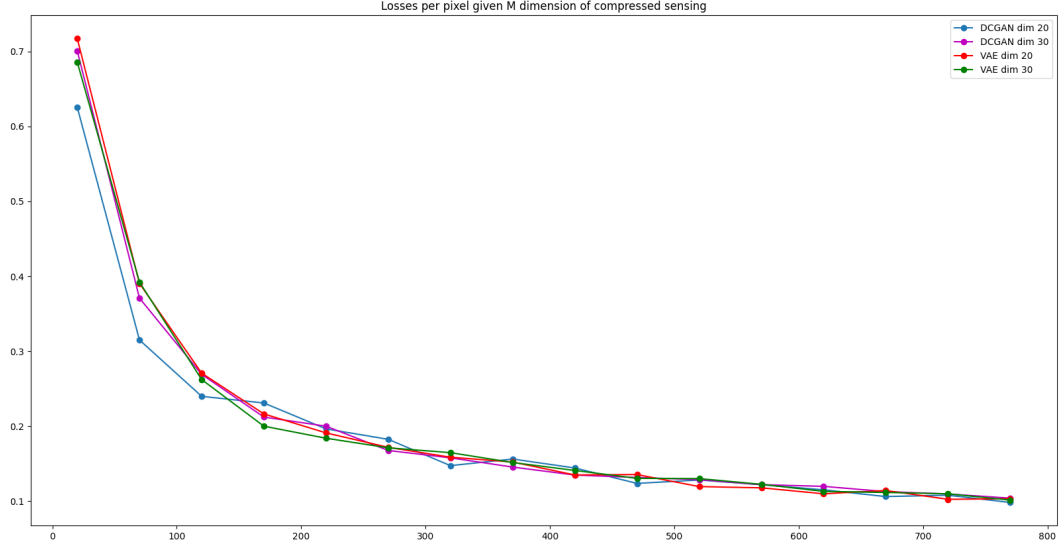


Figure 6.5: Losses per pixel using generative models

To ensure a fair comparison of the models' behavior, we trained them on the same machine for a similar number of epochs.

From the graph above, we can conclude that all the involved generative networks perform significantly better in terms of loss per pixel with respect to the reconstruction obtained with LASSO (figure 6.2 for reference). Among the four of them, the DCGAN with latent dimension equal to 20 is the one that performs better with few measurements, while as the number of measurements increases we can notice that the models tend to achieve the same performances, settling down on an loss of 0.1 - 0.2 after around 400 measurements.

DCGAN is a suitable choice for compressed sensing in a scenario in which high precision and few measurements are required.

On the other hand, taking into consideration the amount of time needed for

training, that in the case of DCGAN is almost 11 times higher than VAE, if the slightly better performance of DCGAN is negligible and it is preferred a less time and resources consuming alternative, than VAE proves to be a valid solution.

7 CONCLUSIONS

This project aimed to assess the performance of compressed sensing using two deep learning models, VAE and GAN, on the MNIST dataset. The models' efficacy in image reconstruction from highly compressed measurements was evaluated and compared to traditional methods like LASSO.

Key Findings:

1. Deep Learning Models Enhance Reconstruction:

- Both VAE and GAN significantly outperformed traditional methods in terms of reconstruction quality with fewer measurements.
- VAEs achieved comparable performance to LASSO with substantially fewer measurements (e.g., 25 vs. 400).
- GANs performed slightly better than VAEs, showing improved reconstruction as the number of measurements increased, though benefits plateaued after around 400 measurements.

2. Generative Models in Compressed Sensing:

- The success of these models in reconstructing images from minimal data showcases the potential of integrating generative models with compressed sensing techniques.
- The ability of these models to leverage learned data distributions enables high-quality reconstructions even in resource-constrained environments.

Future Directions:

The promising results suggest several avenues for further research:

- **Optimizing Measurement Matrices:** Future work could explore designing or learning measurement matrices tailored to specific datasets and reconstruction tasks.
- **Exploring Alternative Architectures:** Investigating other deep learning architectures and their applicability to compressed sensing could yield further improvements.

- **Applications Beyond MNIST:** Applying these methods to more complex and diverse datasets will test the generalizability and robustness of the models.

In conclusion, integrating deep learning with compressed sensing provides a powerful approach for efficient data reconstruction. The demonstrated performance improvements open up new possibilities for applications in various fields, such as medical imaging, remote sensing, and beyond. The integration of these techniques can lead to better utilization of limited data, enhancing the capabilities of modern imaging systems.